



Rubén García
Fernando Lobera
Eduardo Baltrons

Índice

Parte I. Descripción General

- 1.1. Descripción del proyecto
- 1.2. Objetivo general y justificación / Objetivos
- 1.3. Esquema general de funcionamiento
- 1.4. Funcionalidades principales / Alcance
- 1.5. Análisis de la competencia
- 1.6. Análisis DAFO
- 1.7. Marco tecnológico

Parte 2 Análisis y Diseño técnico

2.1.1 Requisitos funcionales

- Módulo de Gestión de Usuarios y Seguridad
- Módulo de Gestión de Grupos (Salas de Estudio)
- Módulo de Gestión de Recursos (Repositorio)
- Módulo de Comunicación y Tiempo Real

2.1.2 Requisitos no funcionales.

- Rendimiento y Eficiencia
- Seguridad y Privacidad
- Usabilidad e Interfaz
- Restricciones Tecnológicas y de Implementación

Parte 3 Planificación y memoria económica

Parte 4 Áreas de gestión del proyecto

- 4.1. Gestión de riesgos
- 4.2. Gestión del equipo del proyecto (personal)

Parte 5 Implementación y pruebas

- 5.1.1. Grado de consecución producto

Parte 6 Innovación

Parte 7 Resultados y conclusiones

Parte 8 Bibliografía

Parte I. Descripción General

1.1. Descripción del proyecto

El proyecto consiste en el diseño, desarrollo e implementación de una **Plataforma Web de Estudio Colaborativo**. Esta herramienta surge de la necesidad de agrupar en un único sitio el intercambio de material didáctico y la comunicación fluida entre estudiantes. A diferencia de las redes sociales convencionales los LMS (*Learning Management Systems*), esta solución se enfoca en la interacción humana.

La plataforma permite a los usuarios registrarse y crear o unirse a "Salas de Estudio" categorizadas por asignaturas. Dentro de estas salas, los usuarios disponen de herramientas para compartir apuntes y bibliografía de forma, así como un canal de comunicación (chat de texto/post estilo foro) y de repositorios de archivos. El sistema está diseñado bajo una arquitectura de **Single Page Application (SPA)**, garantizando una experiencia de usuario gratificante y dinámica.

Uno de los retos que el proyecto aborda es la interactividad en tiempo real y la gestión de contenidos multimedia. Se implementará un sistema de roles para asegurar la moderación de los grupos.

1.2. Objetivo general y justificación / Objetivos

Objetivo General Desarrollar una aplicación web escalable que fomente el aprendizaje colaborativo, proporcionando a los estudiantes un sitio en la web donde puedan organizar grupos de estudio, compartir conocimiento y comunicarse en tiempo real, mejorando así su rendimiento académico y reduciendo el aislamiento en el estudio remoto.

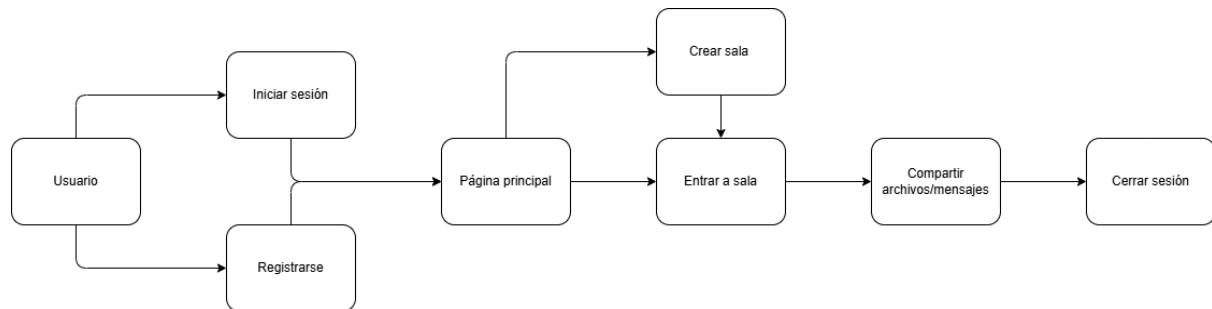
Justificación: La **motivación** principal de este proyecto surge del auge de la educación híbrida y a distancia. Las herramientas actuales suelen estar fragmentadas: los alumnos usan WhatsApp para hablar y Google Drive para archivos, lo que dispersa la información. Este proyecto se justifica por la necesidad de **unificar estas funcionalidades** en un ecosistema diseñado pedagógicamente para el estudio, optimizando el tiempo del alumno y facilitando el acceso a recursos compartidos.

Objetivos Específicos

1. **Implementar un sistema de autenticación seguro:** Garantizar la privacidad y seguridad de los datos de los usuarios mediante el uso de tokens (JWT) y cifrado de contraseñas, permitiendo la gestión de perfiles personalizados.
2. **Desarrollar un módulo de gestión de recursos:** Crear una estructura de base de datos eficiente que permita almacenar, categorizar y recuperar archivos y enlaces compartidos dentro de cada grupo de estudio.
3. **Integrar comunicación en tiempo real:** Implementar Mercure para habilitar funcionalidades de chat instantáneo y notificaciones push, elementos críticos para la coordinación de grupos.

4. **Diseñar una interfaz intuitiva y responsiva:** Utilizar librerías de componentes modernos para asegurar que la plataforma sea accesible desde distintos dispositivos, priorizando la usabilidad (UX/UI).
5. **Excluir dispositivos móviles:** Usualmente a la hora de estudiar los móviles son una distracción y por lo general los estudiantes intentan evitarlos. Aún existiendo la posibilidad de usar la aplicación web desde el dispositivo móvil, está priorizado el uso de la versión de escritorio.

1.3. Esquema general de funcionamiento



1.4. Funcionalidades principales / Alcance

El alcance del proyecto se estructura en tres módulos funcionales principales, cubriendo las necesidades básicas de la gestión colaborativa:

1. Gestión de Usuarios y Accesos:

- Registro e inicio de sesión seguro (Login/Logout).
- Edición de perfil de usuario (biografía, estudios).
- Recuperación de credenciales básica.
- Sistema de roles (Administrador de la sala vs. Usuario estándar).

2. Gestión de Grupos y Recursos (Core):

- Creación, edición y eliminación de grupos de estudio (CRUD).
- Búsqueda y filtrado de grupos por nombre.
- Subida y descarga de archivos (PDFs, imágenes, resúmenes).
- Compartir enlaces externos de interés.
- Organización de recursos dentro del grupo.

3. Comunicación y Tiempo Real:

- Chat grupal integrado en cada sala de estudio.
- Notificaciones en tiempo real.
- Historial de mensajes persistente.

Exclusiones (Fuera del alcance):

- No se implementará un sistema de videoconferencia nativo (se permitirá compartir enlaces a Zoom/Teams, pero el vídeo no se procesa en la plataforma).

- No se desarrollará una aplicación móvil nativa (Android/iOS), aunque la web será *responsive* (adaptable a móviles).
- No se incluye pasarela de pagos ni funciones "Premium".

1.5. Análisis de la competencia

En el mercado existen herramientas consolidadas, aunque pocas combinan todas las funcionalidades específicas para estudiantes en una sola interfaz gratuita:

- **Discord:**
 - *Descripción:* Plataforma de chat y voz muy popular entre jóvenes.
 - *Comparativa:* Aunque es excelente para la comunicación en tiempo real (ventaja), su gestión de archivos e "hilos" académicos es caótica y poco estructurada para el estudio serio (desventaja frente a nuestro proyecto).
- **Moodle / Blackboard:**
 - *Descripción:* Plataformas institucionales usadas por universidades.
 - *Comparativa:* Son muy rígidas y centradas en la comunicación Profesor-Alumno. Nuestro proyecto fomenta la comunicación Alumno-Alumno, siendo más ágil y menos burocrática.
- **Notion:**
 - *Descripción:* Herramienta de productividad y notas.
 - *Comparativa:* Permite compartir recursos de forma excelente, pero carece de chat en tiempo real integrado y funciones sociales dinámicas (descubrimiento de grupos).

1.6. Análisis DAFO

Análisis de la viabilidad del proyecto mediante la matriz de Debilidades, Amenazas, Fortalezas y Oportunidades:

- **Debilidades (Internas):**
 - *Dependencia de la masa crítica:* Una plataforma social no es útil si no hay suficientes usuarios activos y contenido compartido al inicio.
 - *Limitación de almacenamiento:* El alojamiento de archivos pesados puede requerir recursos de servidor costosos si el proyecto escala mucho.
- **Amenazas (Externas):**
 - *Competencia desleal:* Grandes empresas (Microsoft, Google) podrían integrar funcionalidades similares en sus suites educativas gratuitas.
 - *Seguridad:* Riesgo de subida de archivos maliciosos o contenido con derechos de autor (copyright) por parte de los usuarios.
- **Fortalezas (Internas):**
 - *Tecnología de vanguardia:* El uso de una SPA (Single Page Application) y Mercure ofrece una experiencia de usuario superior a los foros universitarios tradicionales.
 - *Nicho específico:* Al estar diseñada *por y para* estudiantes, la UX (Experiencia de Usuario) está adaptada a flujos de trabajo académicos reales, evitando distracciones.

- **Oportunidades (Externas):**
 - *Digitalización educativa:* La tendencia post-pandemia hacia modelos híbridos hace que estas herramientas sean más necesarias que nunca.
 - *Escalabilidad:* Posibilidad futura de añadir "gamificación" (puntos por ayudar) o monetización mediante tutorías privadas.

1.7. Marco tecnológico

Para la implementación se ha seleccionado un stack tecnológico basado en JavaScript, lo que permite un desarrollo unificado y eficiente:

- **Frontend (Interfaz):**
 - **Tailwind CSS:** Framework de estilos "utility-first" que permite un diseño rápido, consistente y totalmente adaptativo (responsive).
 - **Twig:** Es un "motor" de plantillas moderno, rápido y seguro para PHP, predeterminado para su uso en Symfony.
- **Backend (Servidor):**
 - **Mercure:** Es un protocolo abierto diseñado para publicar actualizaciones desde el servidor a los usuarios en tiempo real, ofreciendo una alternativa moderna y eficiente a WebSocket, basado en Server-Sent Events (para el chat y las notificaciones).
 - **Symfony:** Es un framework web de código abierto basado en PHP, diseñado para desarrollar aplicaciones web robustas, escalables y profesionales. Utilizado el patrón de vista MVC, lo que permite separar la lógica del servidor y la presentación, facilitando el código limpio, mantenible y seguro.
- **Base de Datos:**
 - **Postgresql:** Base de datos relacional orientada a objetos y de código abierto. Se elige por su flexibilidad para guardar datos heterogéneos (perfiles, mensajes de chat, metadatos de archivos).
- **Herramientas de Desarrollo:**
 - **Git/GitHub:** Para el control de versiones y trabajo colaborativo del equipo.
 - **Docker:** Es una plataforma que permite a los desarrolladores crear, implementar y ejecutar aplicaciones dentro de contenedores, los cuales son unidades estandarizadas y ligeras que empaquetan el código de la aplicación junto con todas sus dependencias, bibliotecas y configuraciones necesarias.

Parte 2 Análisis y Diseño técnico

2.1 Requisitos

2.1.1 Requisitos funcionales

Módulo de Gestión de Usuarios y Seguridad

- **Registro de Usuarios:** El sistema debe permitir el registro de nuevos estudiantes solicitando correo electrónico, nombre de usuario y contraseña.
- **Autenticación:** El sistema debe permitir el inicio de sesión (Login) validando las credenciales contra la base de datos.
- **Gestión de Perfil:** El usuario podrá editar su perfil personal, añadiendo información biográfica, estudios en curso y una imagen de avatar.

Módulo de Gestión de Grupos (Salas de Estudio)

- **Creación de Salas:** El usuario podrá crear nuevas "Salas de Estudio", definiendo un nombre, una descripción y la categoría de interés asociado.
- **Búsqueda y Filtrado:** El sistema debe ofrecer un buscador que permite localizar grupos existentes por nombre.
- **Unirse/Salir de Grupos:** El usuario podrá unirse a salas de estudio existentes para acceder a sus contenidos y abandonar las mismas cuando lo desee.
- **Moderación de Grupo:** El creador del grupo (o moderador asignado) podrá eliminar a participantes que incumplan las normas de la sala.

Módulo de Gestión de Recursos (Repositorio)

- **Carga de Archivos:** El sistema debe permitir a los usuarios subir archivos (formatos PDF e imágenes) al repositorio de cada grupo de estudio.
- **Descarga de Recursos:** Los miembros del grupo podrán visualizar y descargar los materiales compartidos por otros pares.
- **Enlaces Externos:** El sistema debe permitir compartir y previsualizar enlaces a recursos externos (webs, vídeos de YouTube, artículos).

Módulo de Comunicación y Tiempo Real

- **Chat Grupal:** El sistema debe proporcionar un chat de texto en tiempo real dentro de cada sala de estudio, visible para todos los miembros activos de dicha sala.

2.1.2 Requisitos no funcionales.

Rendimiento y Eficiencia

- **Reproductor de video:** los enlaces de video serán redirigidos a páginas externas para que la web funcione mejor
- **Arquitectura SPA:** La aplicación debe comportarse como una Single Page Application (SPA), evitando la recarga completa de la página al navegar entre secciones internas para garantizar fluidez.

Seguridad y Privacidad

- **Cifrado de contraseñas:** Las contraseñas de los usuarios no deben almacenarse en texto plano; deben ser encriptadas (hashing) antes de guardarse en la base de datos.
- **Seguridad de Sesión:** La gestión de sesiones debe realizarse mediante tokens seguros (JWT), los cuales tendrán un tiempo de expiración definido.

Usabilidad e Interfaz

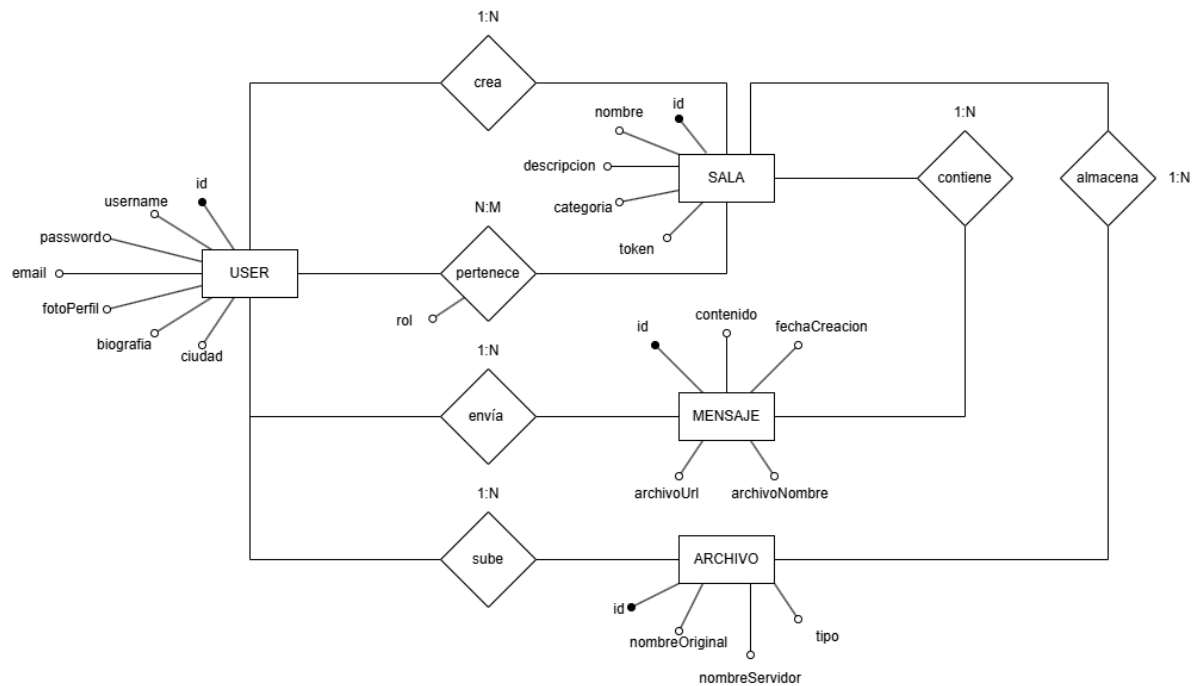
- **Enfoque Desktop:** La interfaz de usuario (UI) estará optimizada prioritariamente para resoluciones de escritorio y portátiles (pantallas grandes), fomentando el estudio inmersivo y minimizando distracciones propias de entornos móviles.
- **Diseño Responsivo:** Aunque el enfoque es de escritorio, la interfaz debe ser responsiva y adaptarse visualmente si el usuario redimensiona la ventana del navegador, sin romper la maquetación.

Restricciones Tecnológicas y de Implementación

- **Stack Tecnológico:** El sistema debe desarrollarse utilizando el stack PostgreSQL para la base de datos, Symfony para la gestión de la web y Mercure para la comunicación en tiempo real.
- **Almacenamiento de Datos:** Se utilizará una base de datos orientada a objetos (Postgresql) para permitir flexibilidad en la estructura de los datos de los grupos y mensajes.
- **Exclusión de Vídeo Nativo:** El sistema no procesa streaming de vídeo ni videoconferencias de forma nativa; estas funcionalidades se delegarán a servicios externos mediante enlaces (Zoom, Teams, etc.).

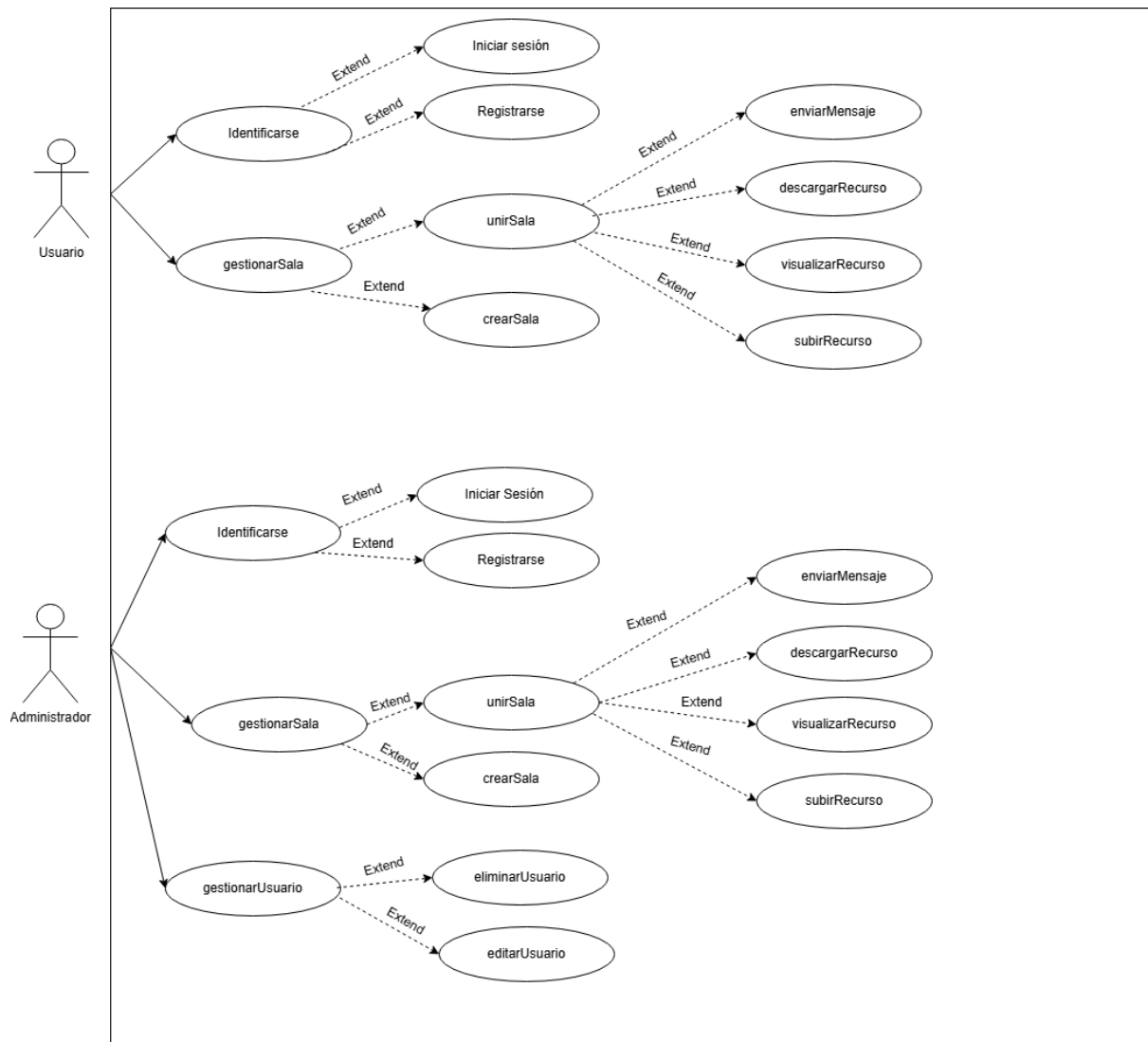
2.1.3. Diseño físico/lógico de datos

2.1.3.1 Diagrama Entidad-Relación



2.1.4 Diseño funcional

2.1.4.1 Casos de Uso



Iniciar Sesión

Nombre: Iniciar Sesión

Descripción: El administrador o usuario solicita hacer el login después de estar registrado

Actor: Administrador y Usuario.

Precondiciones: Se requiere que esté ya registrado.

Curso normal del caso de uso:

1. El administrador o usuario solicita el login
2. El sistema busca si está registrado o no
3. El sistema acepta el login
4. El administrador o usuario tienen acceso

Postcondiciones: El sistema muestra la siguiente pantalla después de registrarse

Alternativas:

5. El sistema no encuentra los datos utilizados ya que no está registrado

Registrarse

Nombre: Registrarse
Descripción: El administrador o usuario solicita hacer el registro para navegar por la web
Actor: Administrador y Usuario.
Precondiciones: Se requiere que no se haya accedido nunca a la página web.
Curso normal del caso de uso: 1. El administrador o usuario solicita el registro 2. El sistema acepta el registro. 3. El administrador o usuario ya tienen acceso y accederán a iniciar sesión
Postcondiciones: El sistema muestra la página de inicio de sesión
Alternativas: 4. El sistema encuentra datos que ya hayan sido utilizados en la web.

Subir Recurso

Nombre: subirRecurso
Descripción: El administrador o usuario suben un recurso para poder utilizarlo
Actor: Administrador o Usuario.
Precondiciones: Se requiere que haya iniciado sesión y este en alguna sala
Curso normal del caso de uso: 1. El administrador o usuario selecciona el archivo que quiere subir. 2. El sistema acepta el archivo y lo sube en la sala . 3. El sistema muestra el archivo subido listo para utilizar
Postcondiciones: El usuario o administrador podrán editarlo, descargarlo y guardarse el archivo
Alternativas: 4. El archivo pesa mucho y no deja subirlo o está en un formato que no está permitido

Descargar Recurso

Nombre: descargarRecurso
Descripción: El administrador o usuario descargan un recurso para poder utilizarlo
Actor: Administrador o Usuario.
Precondiciones: Se requiere que haya iniciado sesión y este en alguna sala
Curso normal del caso de uso: 1. El administrador o usuario selecciona el archivo que quiere descargar. 2. El sistema filtra el archivo y lo descarga..
Postcondiciones: Si el archivo tiene cambios se informará para volverlo a descargar
Alternativas: 3. Abrir el archivo sin necesidad de descargarlo

Visualizar Recurso

Nombre: visualizarRecurso
Descripción: El administrador o usuario pueden visualizar el archivo
Actor: Administrador o Usuario.
Precondiciones: Se requiere que haya iniciado sesión y este en alguna sala
Curso normal del caso de uso: 1. El administrador o usuario selecciona el archivo que quiere visualizar. 2. El sistema acepta el archivo y se abre.
Postcondiciones: Cerrar el archivo una vez haya sido visualizado
Alternativas: 3. Podrás descargar el archivo también para visualizarlo

Enviar Mensaje

Nombre: enviarMensaje
Descripción: El administrador o usuario pueden enviar mensajes
Actor: Administrador o Usuario.
Precondiciones: Se requiere que haya iniciado sesión y este en alguna sala
Curso normal del caso de uso: 1. El administrador o usuario dentro de una sala abren el chat para enviar un mensaje 2. El sistema lo acepta y manda los mensajes que ellos escriban
Postcondiciones: Cerrar el chat una vez finalizada la conversación
Alternativas: 3. Podrás recibir y leer mensajes sin necesidad de mandar nada.

Crear Sala

Nombre: crearSala
Descripción: El administrador o usuario pueden crear una sala para estar con otros usuarios
Actor: Administrador o Usuario.
Precondiciones: Se requiere haber iniciado sesión
Curso normal del caso de uso: 1. El administrador o usuario inicia sesión 2. El administrador o usuario crean la sala con ciertas características 3. El sistema acepta la creación de la sala 4. El administrador o usuario tendrán la sala creada lista para trabajar
Postcondiciones: Abandonar la sala al finalizar
Alternativas: 5. Podrá buscar antes de crear la sala a ver si hay alguna otra sala que te interese.

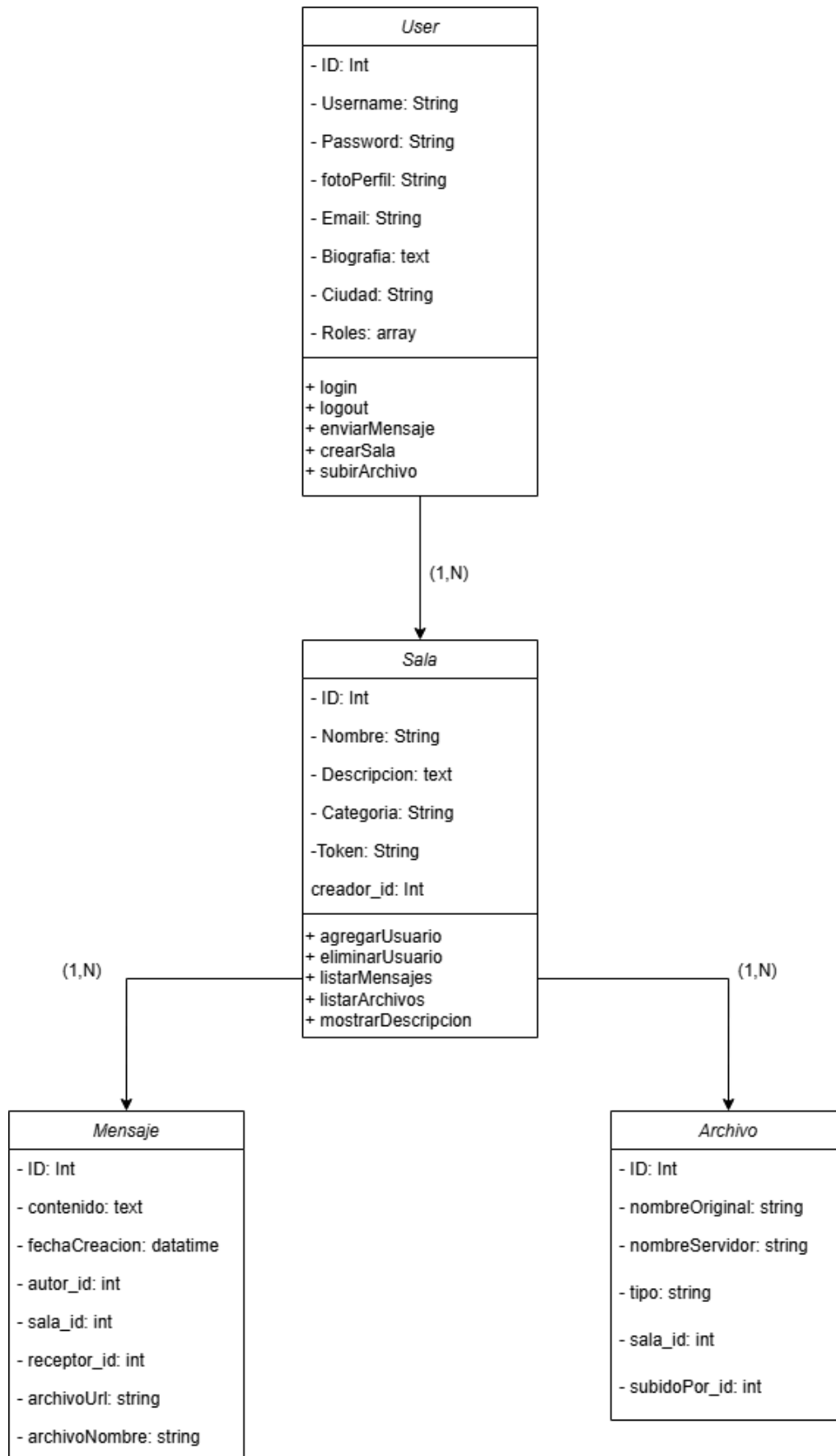
Eliminar Usuario

Nombre: eliminarUsuario
Descripción: El administrador podrá eliminar cualquier usuario
Actor: Administrador.
Precondiciones: Se requiere que sea el propio administrador.
Curso normal del caso de uso: 1. El administrador selecciona al usuario. 2. El administrador selecciona eliminar usuario. 3. El sistema busca el usuario que el administrador quiera eliminar. 4. El sistema elimina ese usuario.
Postcondiciones: El usuario eliminado no podrá volver a usar su cuenta para acceder
Alternativas: 5. El administrador en vez de eliminar al usuario le puede dar un toque de atención.

Editar Usuario

Nombre: editarUsuario
Descripción: El administrador podrá editar cualquier usuario
Actor: Administrador.
Precondiciones: Se requiere que sea el propio administrador.
Curso normal del caso de uso: 1. El administrador selecciona al usuario. 2. El administrador selecciona editar usuario. 3. El sistema busca el usuario que el administrador quiera editar. 4. El administrador edita algún atributo de ese usuario. 5. El sistema guarda los cambios.
Postcondiciones: El usuario sabrá que se le ha editado su propio perfil.
Alternativas: 6. El administrador avisa al usuario para que lo edite el mismo antes de que lo haga el propio administrador.

2.1.4.2 Diagrama de Clases



2.1.4.3 Diagramas de Secuencias

Diagrama de secuencia de la creación de salas.

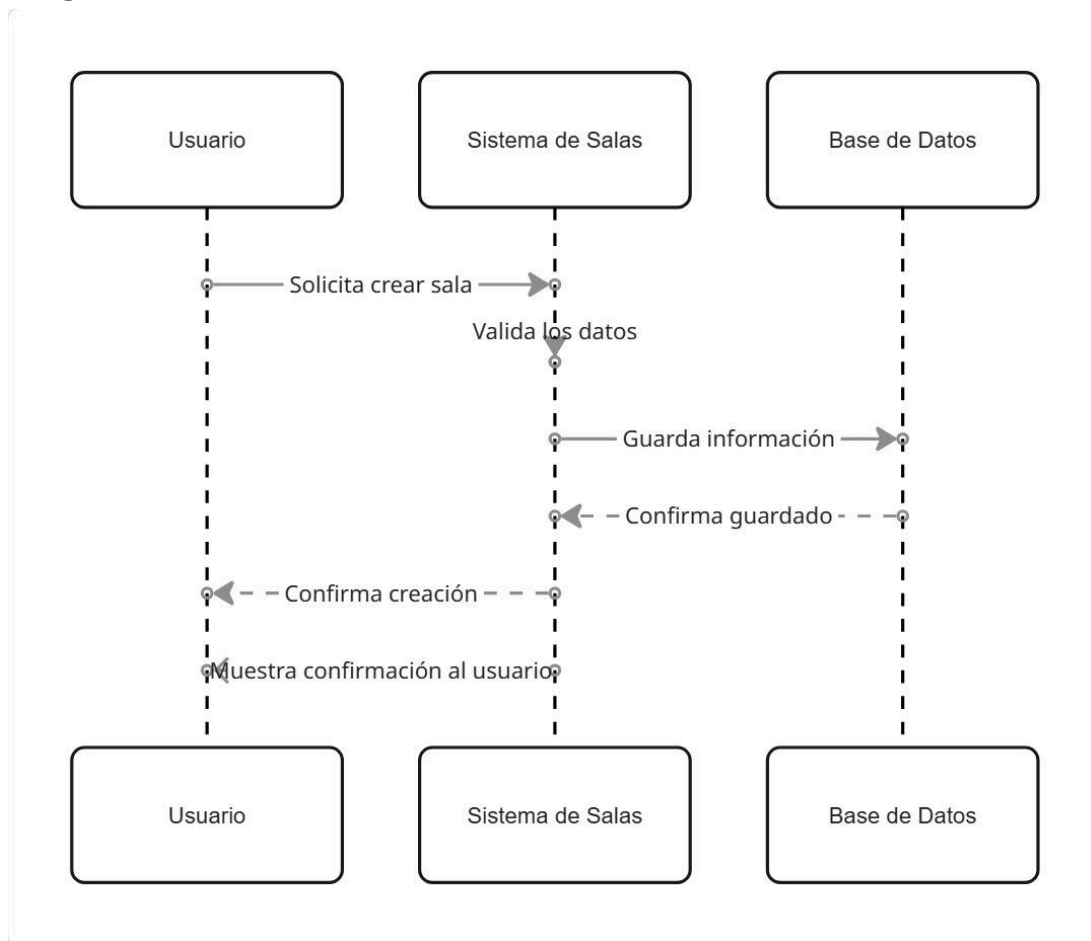


Diagrama de secuencia de la subida de un archivo.

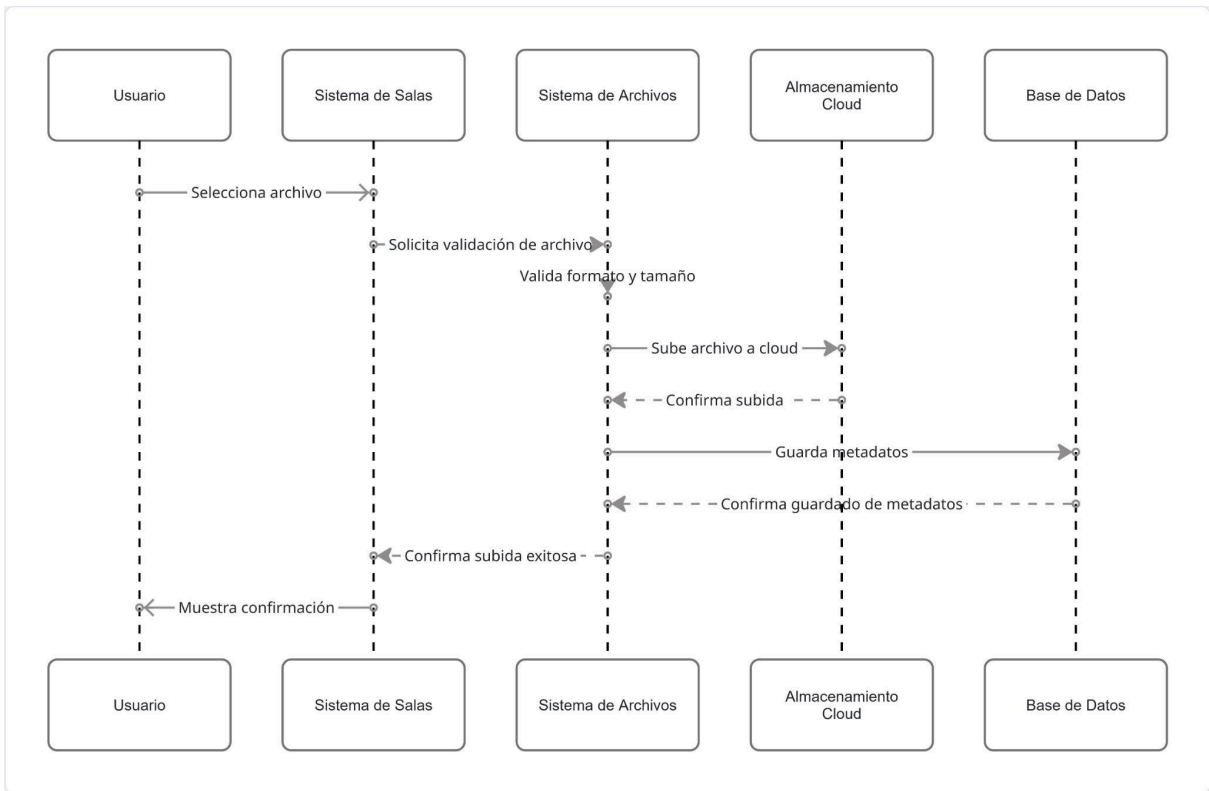


Diagrama de secuencia de visualizar un archivo

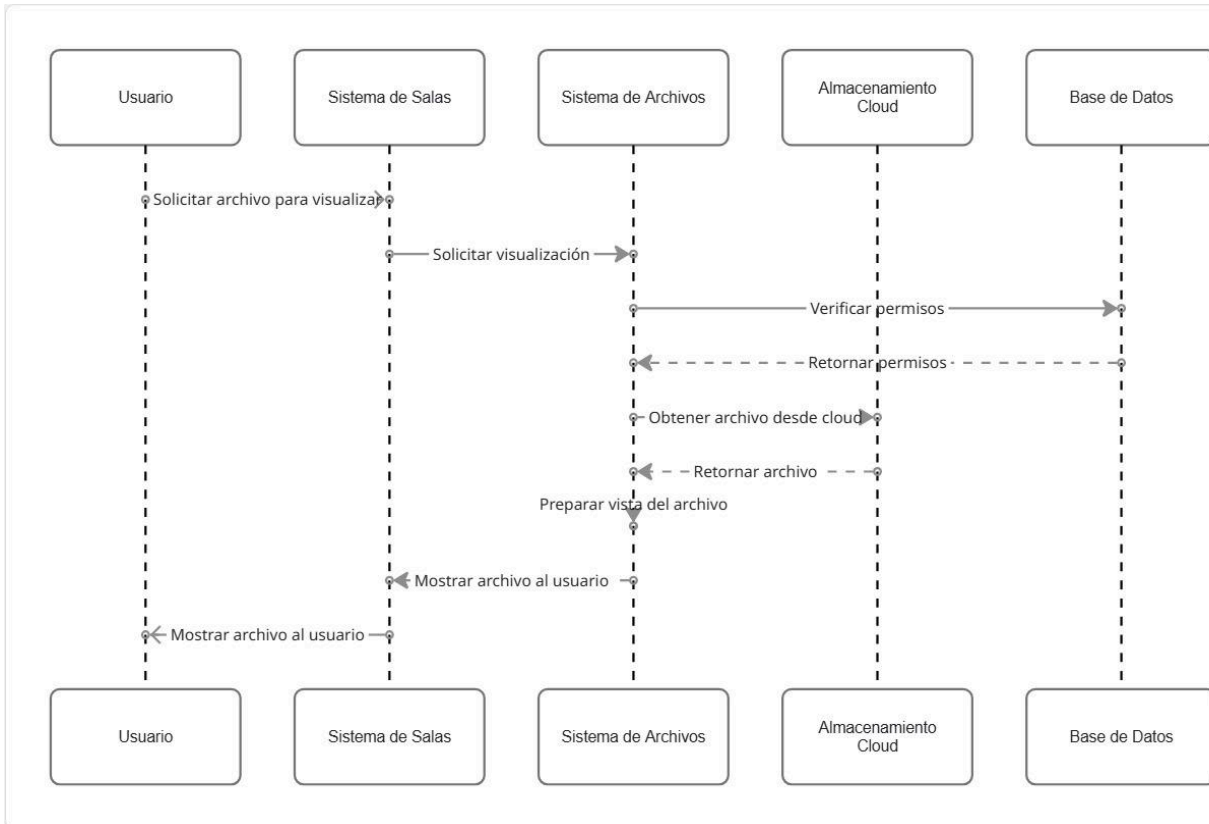


Diagrama de secuencia de descargar un archivo

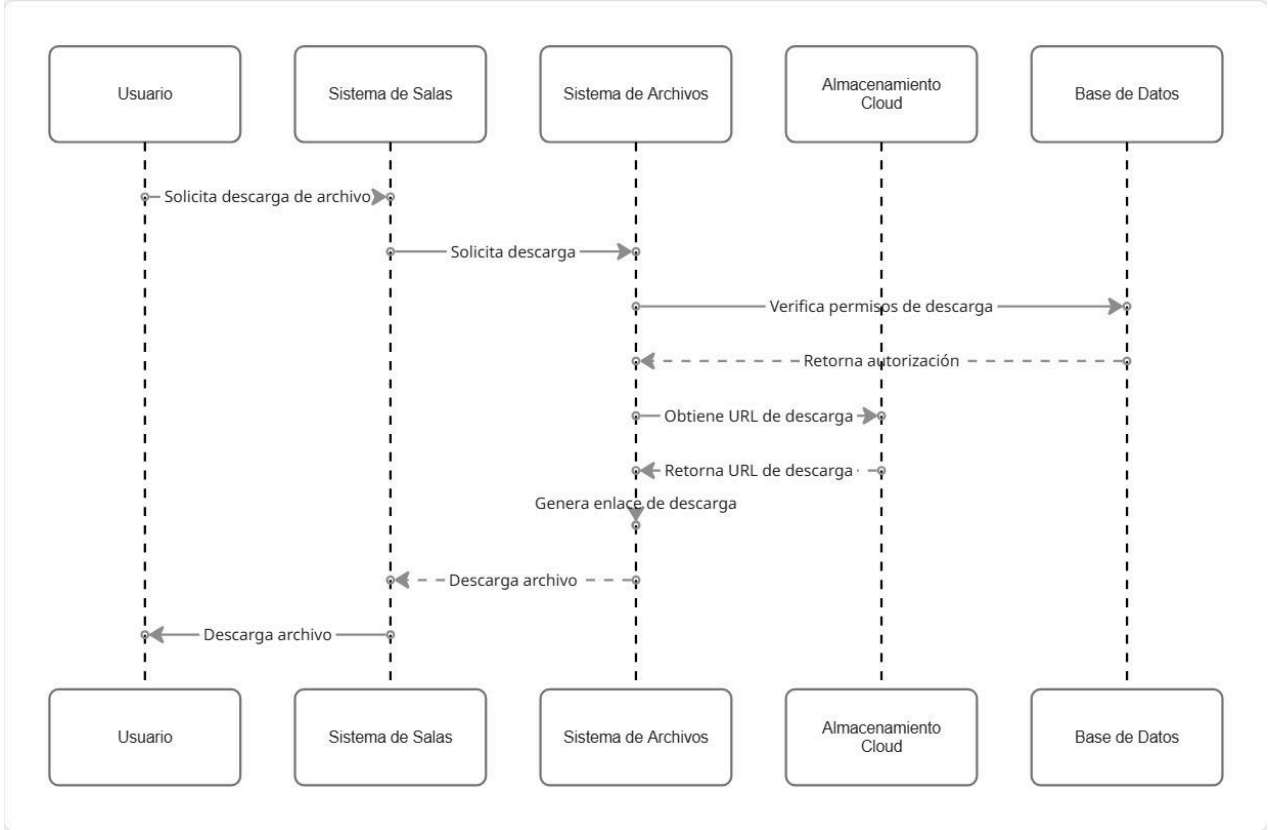
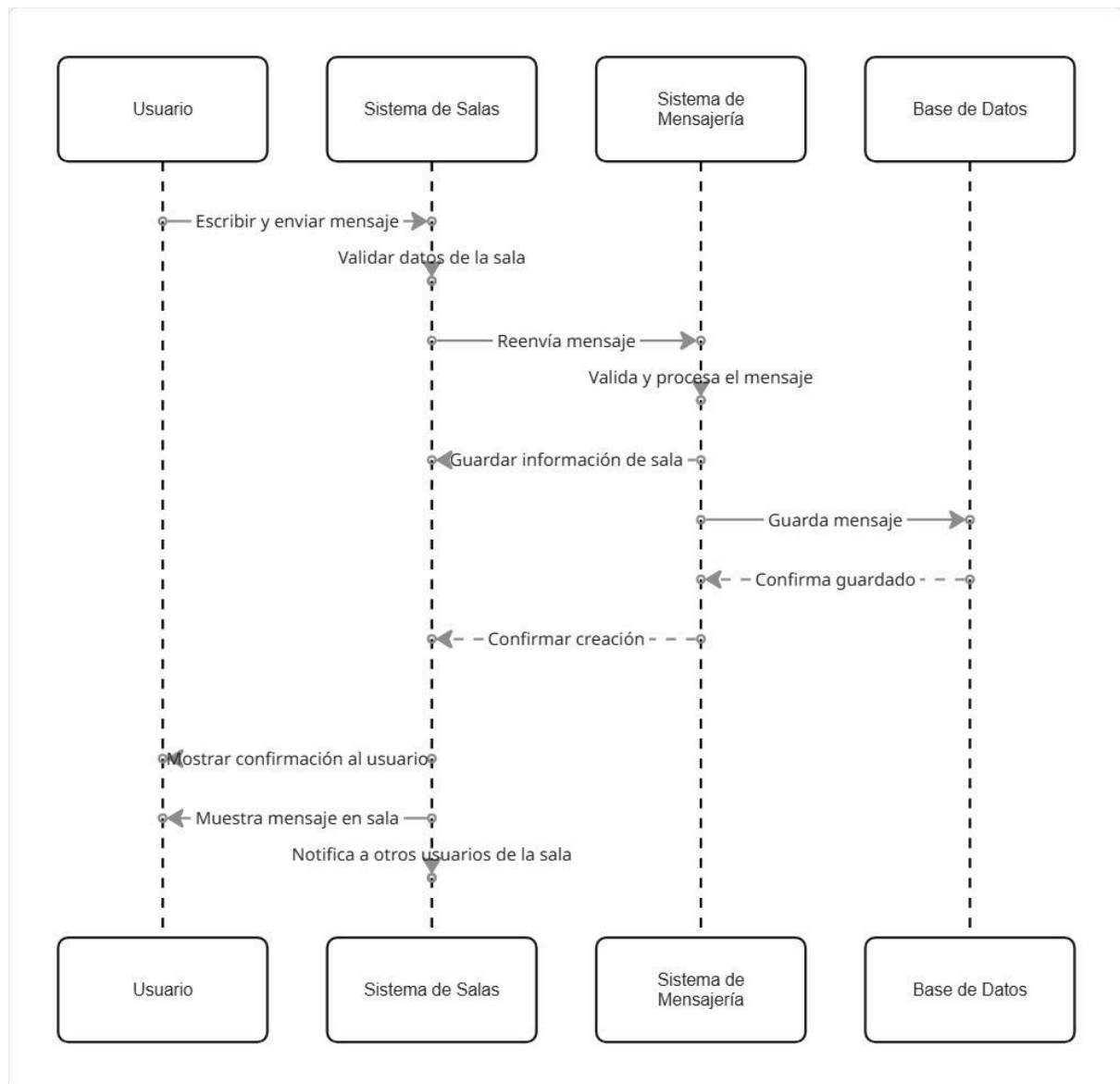


Diagrama de secuencia de enviar un mensaje



Parte 3 Planificación y memoria económica

Para el desarrollo de la Plataforma Web de Estudio Colaborativo, la planificación se ha estructurado en fases secuenciales y ágiles, estimando una duración total de **12 semanas**. A continuación, se presenta una tabla representativa del cronograma (diagrama de Gantt lógico) y la valoración económica.

Planificación de Tareas (Estimación temporal)

Fase	Tareas Principales	Duración	Recursos Necesarios

1. Análisis y Diseño	Toma de requisitos, diseño de BBDD (Postgresql), diseño de interfaces (Figma) y arquitectura.	Semanas 1 - 2	Analista/Diseñador, Figma, draw.io.
2. Desarrollo Backend	Configuración: Symfony, Mercure, autenticación JWT y configuración de Postgresql.	Semanas 3 - 5	Desarrollador Backend, PHPStorm
3. Desarrollo Frontend	Maquetación con Twig y Tailwind CSS, integración de SPA, rutas privadas.	Semanas 6 - 8	Desarrollador Frontend, PHPStorm.
4. Integración Tiempo Real	Implementación de Mercure para chat y notificaciones.	Semanas 9 - 10	Equipo Full-Stack, PHPStorm
5. Pruebas y Despliegue	Testing unitario e integración, corrección de bugs, despliegue en servidor.	Semanas 11 - 12	Docker, Railway y NGrok

Memoria Económica

La siguiente tabla refleja los costes estimados de personal y recursos de infraestructura para la fase de desarrollo:

Concepto / Rol	Horas Estimadas	Coste/Hora (€)	Coste Total (€)

Jefe de Proyecto / Analista	60	35	2.100
Desarrollador Frontend	120	25	3.000
Desarrollador Backend	120	25	3.000
Infraestructura y Software (Cloud, dominios)	N/A	N/A	300
TOTAL ESTIMADO	300 horas		8.400 €

Análisis de monetización y mantenimiento: El coste de mantenimiento mensual (servidores, base de datos en la nube) se estima bajo en la fase inicial. Dado que el proyecto no incluye funciones "Premium" inicialmente, la monetización a medio plazo podría explorarse mediante patrocinios de instituciones educativas, publicidad no intrusiva de material académico, o la futura implementación de un modelo *freemium* para tutorías privadas, lo cual amortizará la inversión inicial de 8.400 €.

Parte 4 Áreas de gestión del proyecto

4.1. Gestión de riesgos

Identificar y mitigar los riesgos es crucial para evitar desviaciones en la planificación o el presupuesto del proyecto.

Riesgo	Probabilidad	Impacto	Plan de Contingencia

Retrasos en la integración de Mercure	Media	Alto	Asignar más horas en las semanas 9-10; utilizar librerías simplificadas de Mercure o reducir el alcance del chat a recarga asíncrona si el tiempo es crítico.
Brechas de seguridad en subida de archivos	Baja	Alto	Restringir por backend (Express) los tipos de archivos permitidos (solo PDF e imágenes), limitar el tamaño máximo a 10MB y utilizar almacenamiento seguro.
Baja adopción de usuarios al inicio	Alta	Medio	Crear grupos "semilla" con contenido precargado (apuntes reales de asignaturas comunes) para que los primeros usuarios encuentren valor inmediato.
Sobrecarga de la base de datos (Postgresql)	Baja	Alto	Implementar paginación en el historial del chat y en la carga de recursos desde el primer día para no saturar las peticiones.

4.2. Gestión del equipo del proyecto (personal)

El proyecto cuenta con una estructura ágil orientada al desarrollo web moderno. Las funciones se distribuyen de la siguiente manera:

- **Jefe de Proyecto / Scrum Master:** Encargado de velar por el cumplimiento del cronograma, gestionar los riesgos y actuar como puente entre los requisitos funcionales y el equipo técnico.
- **Desarrollador Backend:** Responsable de la arquitectura del servidor, la seguridad de la web, la configuración de Postgresql y la lógica de Mercure.
- **Desarrollador Frontend:** Encargado de plasmar el diseño UI/UX en código utilizando Tailwind CSS, asegurando la responsividad y el comportamiento SPA.
- **QA / Tester (Transversal):** Encargado de realizar pruebas de estrés, pruebas de los *endpoints* plasmados en Symfony y validar la usabilidad de la interfaz.

Parte 5 Implementación y pruebas

El uso de Tailwind y Twig aceleró significativamente el desarrollo de la interfaz de usuario sin necesidad de escribir hojas de estilo complejas desde cero.

Se llevaron a cabo pruebas exhaustivas en dos niveles:

1. **Pruebas de Integración (Mercure):** Simulaciones de múltiples usuarios conectados simultáneamente a una misma "Sala de Estudio" para verificar que los mensajes del chat y las notificaciones llegarán en tiempo real sin latencia perceptible.
2. **Pruebas de Usuario (Caja negra):** Navegación completa por la plataforma simulando el flujo de un estudiante real (Registro -> creación de grupo -> subida de un PDF/imagen -> envío de mensaje).

5.1.1. Grado de consecución producto

El producto final ha alcanzado un alto grado de cumplimiento respecto a la planificación inicial.

- **Comparativa Gantt:** El proyecto se ajustó en un 90% a los tiempos previstos. Hubo un ligero retraso durante la integración de Mercure, pero se compensa en la fase de pruebas.
- **Requisitos funcionales:** Se ha logrado implementar el 100% del núcleo funcional (Core): autenticación, gestión de salas, subida de recursos básicos y chat en tiempo real.
- **Prototipo:** Existe un prototipo funcional desplegado de manera local/cloud que demuestra todas las interacciones.
- **Mapa de navegación:** La navegación de la SPA fluye sin recargas, partiendo de un *Dashboard* central hacia las vistas de *Detalle de Sala*, *Perfil* y *Buscador*, tal como se diseñó en las interfaces previas.

Parte 6 Innovación

La innovación principal reside en que el enfoque del proyecto es la interacción de los usuarios. Mientras que la mayoría de aplicaciones educativas intentan ser todo en uno.

El uso de **Mercure** nos permite que la comunicación entre el cliente y el servidor sea bidireccional además elimina la sobrecarga de encabezados HTTP repetidos, reduciendo la latencia y el consumo de recursos elimina la sobrecarga de encabezados HTTP repetidos, reduciendo la latencia y el consumo de recursos

Funcionalidades potenciales a futuro:

- Implementación de Inteligencia Artificial (ej. resúmenes automáticos de los PDFs subidos al grupo).
- Sistema de gamificación académica (insignia por ser el usuario que más apuntes comparte o más dudas resuelve en el chat).

Parte 7 Resultados y conclusiones

El desarrollo de la Web ha demostrado que es posible crear un entorno digital que favorezca el intercambio de material didáctico y la comunicación fluida entre estudiantes sin depender de aplicaciones generalistas.

Como conclusión principal queremos decir que este proyecto nos ha ayudado mucho a consolidar conocimientos avanzados de desarrollo web Full-Stack, sino que también nos ha ayudado desarrollar mejor nuestras soft skills como la comunicación, el trabajo en equipo etc...

El resultado es una aplicación rápida, segura y altamente escalable gracias a la naturaleza de las tecnologías que hemos usado.

Y para finalizar, es una solución real, viable y muy útil para una gran cantidad de personas.

Parte 8 Bibliografía

- **Postgresql Inc.** (s.f.). Postgresql [documentación](#).
- **Symfony.** [web de symfony](#)
- **Mercure.** <https://symfony.com/doc/current/mercure.html>
- **Tailwind Labs.** (s.f.). *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML*. Recuperado de <https://tailwindcss.com/>
- **Auth.** (s.f.). *JSON Web Tokens (JWT) Introduction*. Recuperado de <https://jwt.io/introduction/>